



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Тестирование и верификация программного обеспечения»

Практическая работа № 4
АНАЛИЗАТОРЫ КОДА

Студенты группы *ИКБО-50-23, Астахов С.П.*

(подпись)

Преподаватель *Ильичев Г.П.*

(подпись)

Отчёт представлен «___»_____2025 г.

Москва 2025 г.

Содержание

1. ЦЕЛЬ И ЗАДАЧИ.....	4
2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	5
2.1. Проекты.....	5
2.1.1. Проект Матрица(python).....	5
2.1.2. Проект «динамический список»(crr).....	8
2.2. Статический анализ кода.....	13
2.2.1. Статических анализатор проекта «Матрица»(Pylint - Полная проверка качества).....	13
2.2.2. Статический анализатор проекта «Матрица»(randon - Сложность функций).....	14
2.2.3. Статический анализатор проекта «Матрица»(PyCodeStyle - СтилЬ PEP8).....	14
2.2.4. Статический анализатор проекта «динамический список»(Crrpcheck - поиск багов).....	15
2.2.5. Статический анализатор проекта «динамический список»(FlawFinder - Поиск уязвимостей).....	15
2.2.6. Статический анализатор проекта «динамический список»(Crrplint - кодстайл).....	16
2.3. Сравнительный анализ.....	16
2.4. Внесение ошибок в проекты и повторные статический анализы.....	17
2.4.1. Внесение ошибок в проект «Матрица».....	17
2.4.1.1. Статический анализатор кода - pylint.....	18
2.4.1.2. Статический анализатор кода — radon.....	18
2.4.1.3. Статический анализатор кода — pycodestyle.....	19
2.4.2. Внесение ошибок в проект «Динамический список».....	20
2.4.2.1. Статический анализатор кода - crrpcheck.....	21
2.4.2.2. Статический анализатор кода - flawfinder.....	22
2.4.2.3. Статический анализатор кода - crrplint.....	22
2.5. Повторный сравнительный анализ.....	23

2.6. Вывод об эффективности статического анализа на раннем выявлении ошибок.....	24
2.7. Динамический анализ кода.....	24
2.7.1. Динамический анализатор проекта «Матрица»(cProfile).....	27
2.7.2. Динамический анализатор проекта «Матрица»(memory-profiler).....	28
2.7.3. Динамический анализатор проекта «Динамический список»(Valgrind)	29
2.7.4. Динамический анализатор проекта «Динамический список»(AddressSanitizer).....	31
2.8. Сравнительный анализ.....	32
2.9. Внесение ошибок в проекты и повторные динамические анализы.....	32
2.9.1. Внесение ошибок в проект «Матрица».....	32
2.9.1.1. Динамический анализатор кода — cProfile.....	35
2.9.1.2. Динамический анализатор кода — memory-profiler.....	35
2.9.2. Внесение ошибок в проект «Динамический список».....	35
2.9.2.1. Динамический анализатор кода — Valgrind.....	38
2.9.2.2. Динамический анализатор кода — AddressSanitizer.....	39
2.10. Повторный сравнительный анализ.....	39
2.11. Вывод об эффективности динамического анализа для ошибок, возникающих в реальных условиях исполнения.....	40
3. АНАЛИЗ И ВЫВОДЫ.....	41

1. ЦЕЛЬ И ЗАДАЧИ.

Цель работы: ознакомиться с основными принципами и методами использования статических и динамических анализаторов кода для раннего выявления ошибок и потенциальных уязвимостей, что позволит повысить качество, безопасность и надёжность программного обеспечения.

Для достижения поставленной цели работы необходимо выполнить ряд задач:

1. Изучить теоретические основы статического и динамического анализа кода.
2. Ознакомиться с популярными инструментами статического анализа и динамического анализа.
3. Применить выбранные анализаторы к ранее разработанным учебным проектам на ранних языках программирования.
4. Провести анализ исходного кода до и после внесения целенаправленных ошибок, оценить адекватность обнаружения дефектов.
5. Сформировать детальный отчёт с критическим анализом результатов, выводами о преимуществах и ограничениях каждого подхода.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Проекты

2.1.1. Проект Матрица(python)



```
Matrix.py U x  G dynamic_list.cpp U  C dynamic_list.h U  E radon_complexity.txt U
Matrix.py > Matrix > create_matrix
1  class Matrix:
2      def __init__(self, matrix_v=[], size="1x1"):
3          self.matrix_v = matrix_v
4          if len(matrix_v) > 0:
5              self.size = f"{len(matrix_v)}x{len(matrix_v[0])}"
6          else:
7              self.size = size
8
9      def change_element(self, index_row, index_col, element):
10         if (index_row >= len(self.matrix_v) or
11             index_col >= len(self.matrix_v[0])):
12             raise IndexError(
13                 f"Индекс [{index_row}][{index_col}] "
14                 f"вне границ матрицы {self.size}"
15             )
16         self.matrix_v[index_row][index_col] = element
17
18     def create_matrix(self, size="3x3", matrixDefault=True) -> list:
19         if not self._validate_size_format(size):
20             raise ValueError(
21                 f"Неверный формат размера: {size}. Используйте формат 'NxM'"
22             )
23
24         self.size = size
25         self.matrix_v = []
26
27         if matrixDefault:
28             max_var = 1
29             for i in range(int(self.size[0])):
30                 matrix_prom = list()
31                 for j in range(int(self.size[2])):
32                     matrix_prom.append(max_var)
33                     max_var += 1
34                 self.matrix_v.append(matrix_prom)
35             return self.matrix_v
36
37         for i in range(int(self.size[0])):
38             matrix_prom = list()
```

Рисунок 1 - Matrix.py(Часть 1)

```

39         for j in range(int(self.size[2])):
40             value = int(input(
41                 f"Введите значение для элемента[{i + 1}][{j + 1}]:"
42             ))
43             matrix_prom.append(value)
44             self.matrix_v.append(matrix_prom)
45         return self.matrix_v
46
47     @staticmethod
48     def _validate_size_format(size):
49         # Проверяет корректность формата размера
50         return (
51             len(size) == 3
52             and size[1] == "x"
53             and size[0].isdigit()
54             and size[2].isdigit()
55         )
56
57     @staticmethod
58     def sum(matrix1, matrix2) -> "Matrix":
59         # Проверка на пустые матрицы
60         if not matrix1.matrix_v or not matrix2.matrix_v:
61             raise ValueError("Одна из матриц пуста")
62
63         # Проверка совпадения размеров
64         if matrix1.size != matrix2.size:
65             raise ValueError(
66                 f"Размеры матриц не совпадают: {matrix1.size} и {matrix2.size}"
67             )
68
69         # Проверка, что все элементы являются числами
70         for i in range(len(matrix1.matrix_v)):
71             for j in range(len(matrix1.matrix_v[0])):
72                 if not isinstance(
73                     matrix1.matrix_v[i][j], (int, float)
74                 ) or not isinstance(matrix2.matrix_v[i][j], (int, float)):
75                     raise TypeError("Элементы матриц должны быть числами")
76
77         matr1 = [row[:] for row in matrix1.matrix_v]

```

Рисунок 2 - Matrix.py(Часть 2)

```

77     matr1 = [row1] + row2 + matr1[2*2:2*2+1]
78     matr2 = matrix2.matrix_v
79
80     for row in range(len(matr2)):
81         for col in range(len(matr2[0])):
82             matr1[row][col] += matr2[row][col]
83     return Matrix(matr1)
84
85     def transposition(self) -> "Matrix":
86         if not self.matrix_v:
87             raise ValueError("Матрица пуста, транспонирование невозможно")
88
89         rows = len(self.matrix_v)
90         cols = len(self.matrix_v[0])
91
92         # Создаем новую транспонированную матрицу
93         transposed = []
94         for j in range(cols):
95             new_row = []
96             for i in range(rows):
97                 new_row.append(self.matrix_v[i][j])
98             transposed.append(new_row)
99
100        return Matrix(transposed)
101
102    @staticmethod
103    def multiplication(matrix1, matrix2) -> "Matrix":
104        # Проверка на пустые матрицы
105        if not matrix1.matrix_v or not matrix2.matrix_v:
106            raise ValueError("Одна из матриц пуста")
107
108        # Проверка совместимости размеров
109        if len(matrix1.matrix_v[0]) != len(matrix2.matrix_v):
110            raise ValueError(
111                f"Несовместимые размеры для умножения: "
112                f"{matrix1.size} и {matrix2.size}"
113            )
114
115        # Проверка, что все элементы являются числами
116        for i in range(len(matrix1.matrix_v)):

```

Рисунок 3 - Matrix.py(Часть 3)

```

117         for j in range(len(matrix1.matrix_v[0])):
118             if not isinstance(matrix1.matrix_v[i][j], (int, float)):
119                 raise TypeError(
120                     f"Элемент matrix1[{i}][{j}] = "
121                     f"{matrix1.matrix_v[i][j]} не является числом"
122                 )
123
124         for i in range(len(matrix2.matrix_v)):
125             for j in range(len(matrix2.matrix_v[0])):
126                 if not isinstance(matrix2.matrix_v[i][j], (int, float)):
127                     raise TypeError(
128                         f"Элемент matrix2[{i}][{j}] = "
129                         f"{matrix2.matrix_v[i][j]} не является числом"
130                     )
131
132         rows1 = len(matrix1.matrix_v)
133         cols1 = len(matrix1.matrix_v[0])
134         cols2 = len(matrix2.matrix_v[0])
135
136         result = []
137         for i in range(rows1):
138             row = []
139             for j in range(cols2):
140                 sum_val = 0
141                 for k in range(cols1):
142                     sum_val += matrix1.matrix_v[i][k] * matrix2.matrix_v[k][j]
143                 row.append(sum_val)
144             result.append(row)
145
146         return Matrix(result)
147
148     def print(self):
149         if not self.matrix_v:
150             print("Матрица пуста")
151             return
152
153         for row in self.matrix_v:
154             print(row)
155         print(f"Размер: {self.size}")

```

Рисунок 4 - Matrix.py(Часть 4)

Данный проект создаёт матрицу и проделывает с ней некоторые операции с помощью методов класса «Matrix»: print(выводит матрицу), multiplation(перемножает матрицы), transposition(транспонирует матрицу), sum(суммирует две матрицы), create_matrix(создаёт матрицу), change_element(изменение элемента)

2.1.2. Проект «динамический список»(сpp)


```

> .vscode
> myvenv
~lock.4practic... U
4practice.odt U
dinamic_list.cpp U
dinamic_list.h U
Matrix.py U
pycodestyle_re... U
pylint_report.txt U
radon_complex... U
Практическая рабо...
1 #pragma once
2 #include <iostream>
3
4 using std::cout;
5 using std::cin;
6 using std::endl;
7
8 template<typename T>
9 class List{
10
11 public:
12     List();
13     ~List();
14     T& operator[](const int index);
15     int GetSize() { return Size; }; // Получение размера
16     void pop_front(); // Удаление первого элемента
17     void push_front(T data);
18     void create_list(int n);
19     void task1(T a, T b);
20     void task2();
21     void task3();
22
23 private:
24     template<typename T>
25     class Node {
26     public:
27         Node* pNext;
28         T data;
29
30         Node(T data = T(), Node* pNext = nullptr) {
31             this->data = data;
32             this->pNext = pNext;
33         }
34     };
35     int Size;
36     Node<T> *head;
37
38 };
39
40 // В отдельный файл нельзя вынести шаблонные методы класса
41

```

OUTLINE
TIMELINE
Main* 0 0 0

Рисунок 5 - dinamic_list.h(Часть 1)

```

42  template<typename T>
43  List<T>::List() {
44      Size = 0;
45      head = nullptr;
46  }
47
48  template<typename T>
49  List<T>::~~List() {
50      while (Size) {
51          pop_front();
52      }
53  }
54
55  template<typename T>
56  T& List<T>::operator[](const int index) {
57      int counter = 0;
58      Node<T>* current = this->head;
59      while (current != nullptr) {
60
61          if (counter == index)
62          {
63              return current->data;
64          }
65
66          current = current->pNext;
67          counter++;
68      }
69  }
70
71  template<typename T>
72  void List<T>::pop_front() {
73      Node<T>* temp = head;
74      head = head->pNext;
75      delete temp;
76      Size--;
77  }
78
79  template<typename T>
80  void List<T>::push_front(T data) {
81      head = new Node<T>(data, head);
82      Size++;

```

Рисунок 6 - dinamic_list.h(Часть 2)

```

82     Size++;
83 }
84
85 template<typename T>
86 void List<T>::create_list(int n){
87     T data;
88
89     for (int i = 0; i < n; i++) {
90         cout << "Введите данные: "; cin >> data;
91         push_front(data);
92     }
93 }
94
95 template<typename T>
96 void List<T>::task1(T a, T b) {
97     Node<T>* current = head;
98
99     if (current == nullptr || current->pNext == nullptr) {
100         return;
101     }
102
103     while (current->pNext->pNext != nullptr) {
104         current = current->pNext;
105     }
106
107     current->pNext = new Node<T>(a, current->pNext);
108     current->pNext->pNext = new Node<T>(b, current->pNext->pNext);
109     Size += 2;
110 }
111
112 template<typename T>
113 void List<T>::task2() {
114     Node<T>* current = head;
115     Node<T>* prev = nullptr;
116     while (current != nullptr && current->data >= 0) {
117         prev = current;
118         current = current->pNext;
119     }
120
121     if (current != nullptr && prev != nullptr) {

```

Рисунок 7 - dinamic_list.h(Часть 3)

```

121     if (current != nullptr && prev != nullptr) {
122         delete current;
123         Size--;
124     }
125     else if (current != nullptr) {
126         pop_front();
127     }
128 }
129
130 template<typename T>
131 void List<T>::task3() {
132     if (head == nullptr || head->pNext == nullptr) {
133         return; // List is empty or has only one node.
134     }
135     Node<T>* current = head;
136     Node<T>* maxNodePrev = nullptr;
137     Node<T>* maxNode = head;
138     Node<T>* prev = nullptr;
139     T maxValue = head->data;
140
141     while (current != nullptr) {
142         if (current->data > maxValue) {
143             maxValue = current->data;
144             maxNodePrev = prev;
145             maxNode = current;
146         }
147         prev = current;
148         current = current->pNext;
149     }
150
151     if (maxNode->pNext == nullptr) {
152         return;
153     }
154
155     if (maxNodePrev == nullptr) {
156         head = maxNode->pNext;
157     }
158     else {
159         maxNodePrev->pNext = maxNode->pNext;
160     }
161 }

```

Рисунок 8 - dinamic_list.h(Часть 4)

```

155     if (maxNodePrev == nullptr) {
156         head = maxNode->pNext;
157     }
158     else {
159         maxNodePrev->pNext = maxNode->pNext;
160     }
161
162     prev->pNext = maxNode;
163     maxNode->pNext = nullptr;
164 }

```

Рисунок 9 - dinamic_list.h(Часть 5)

Реализованная работа для работы со списком на языке c++. В этом классе содержатся такие методы: operator[] (Получение элемента из [] скобок), GetSize (получить размер списка), pop_front (удалить вначале элемент), push_front (добавить вначале элемент), create_list (создать список), а также реализует выполнение трёх заданий:

task1 - 2 вставки узлов перед последним узлом с заданными значениями,

task2 - Метод, удаление из списка L первое отрицательное значение, если оно присутствует в списке,

task3 - нахождения в списке L максимальное значение и перенести узел с этим значением в конец списка.

2.2. Статический анализ кода

2.2.1. Статический анализатор проекта «Матрица» (Pylint - Полная проверка качества)


```
EXPLORER  ...  ▹ pylint_report.txt U X
4          ▹ pylint_report.txt
1          ***** Module Matrix
2          Matrix.py:149:0: C0305: Trailing newlines (trailing-newlines)
3          Matrix.py:1:0: C0114: Missing module docstring (missing-module-docstring)
4          Matrix.py:1:0: C0103: Module name "Matrix" doesn't conform to snake_case naming style (invalid-name)
5          Matrix.py:1:0: C0115: Missing class docstring (missing-class-docstring)
6          Matrix.py:2:4: W0102: Dangerous default value [] as argument (dangerous-default-value)
7          Matrix.py:9:4: C0116: Missing function or method docstring (missing-function-docstring)
8          Matrix.py:16:4: C0116: Missing function or method docstring (missing-function-docstring)
9          Matrix.py:16:40: C0103: Argument name "matrixDefault" doesn't conform to snake_case naming style (invalid-name)
10         Matrix.py:28:30: R1734: Consider using [] instead of list() (use-list-literal)
11         Matrix.py:36:26: R1734: Consider using [] instead of list() (use-list-literal)
12         Matrix.py:54:4: C0116: Missing function or method docstring (missing-function-docstring)
13         Matrix.py:66:8: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
14         Matrix.py:76:8: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
15         Matrix.py:81:4: C0116: Missing function or method docstring (missing-function-docstring)
16         Matrix.py:99:4: C0116: Missing function or method docstring (missing-function-docstring)
17         Matrix.py:111:8: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
18         Matrix.py:118:8: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
19         Matrix.py:141:4: C0116: Missing function or method docstring (missing-function-docstring)
20
21         -----
22         Your code has been rated at 8.06/10 (previous run: 8.06/10, +0.00)
23
24

TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO
● ~/Тестирование и верификация программного обеспечения/4 Main ● ? ▶ pylint Matrix.py --output-format=text > pylint_report.txt
○ ~/Тестирование и верификация программного обеспечения/4 Main ● ? X ▮
```

Рисунок 10 - Запуск Pylint

2.2.2. Статический анализатор проекта «Матрица»(randon - Сложность функций)

```
EXPLORER  ...  ▹ Matrix.py U  ▹ radon_complexity.txt U X
4          ▹ radon_complexity.txt
1          Matrix.py
2          M 103:4 Matrix.multiplication - C (13)
3          M 58:4 Matrix.sum - C (11)
4          C 1:0 Matrix - B (7)
5          M 18:4 Matrix.create_matrix - B (7)
6          M 48:4 Matrix._validate_size_format - A (4)
7          M 85:4 Matrix.transposition - A (4)
8          M 9:4 Matrix.change_element - A (3)
9          M 148:4 Matrix.print - A (3)
10         M 2:4 Matrix.__init__ - A (2)
11

TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO
● ~/Тестирование и верификация программного обеспечения/4 Main ● ? ▶ radon cc Matrix.py -s > radon_complexity.txt
○ ~/Тестирование и верификация программного обеспечения/4 Main ● ? ▶ ▮
```

Рисунок 11 - Запуск radon

2.2.3. Статический анализатор проекта «Матрица»(PyCodeStyle - СтилЬ PEP8)

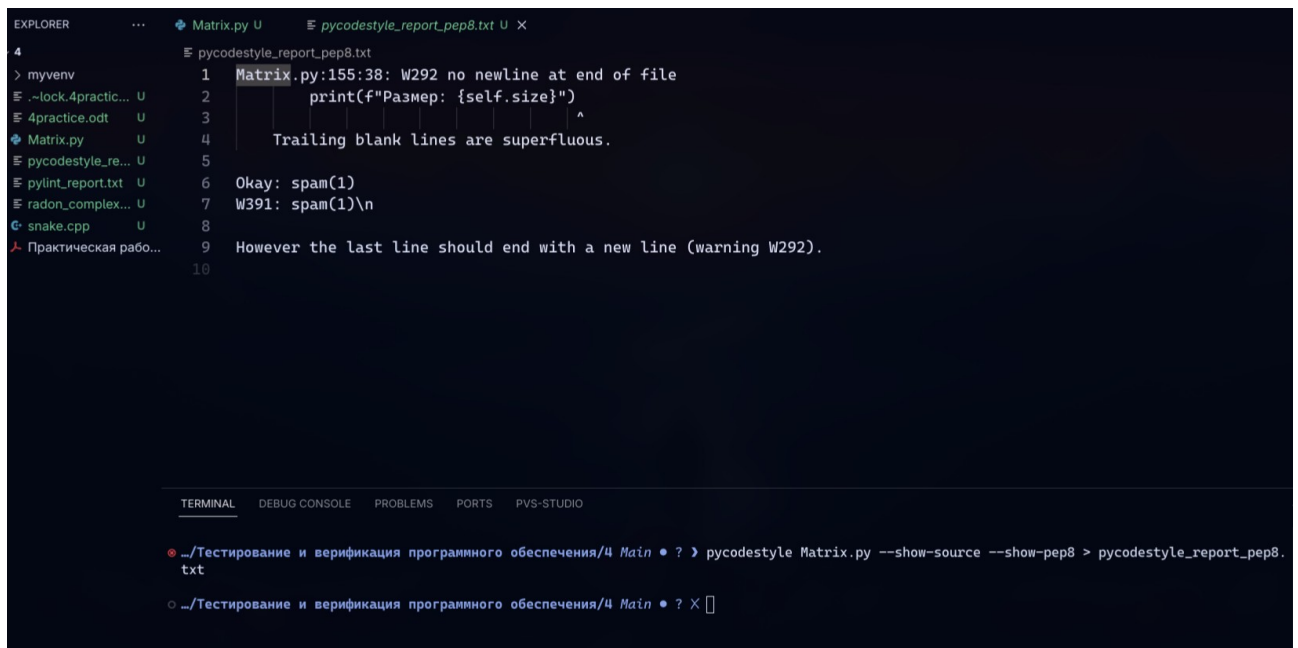


Рисунок 12 - Запуск pycodestyle

2.2.4. Статический анализатор проекта «динамический список»(Cpptest - поиск багов)

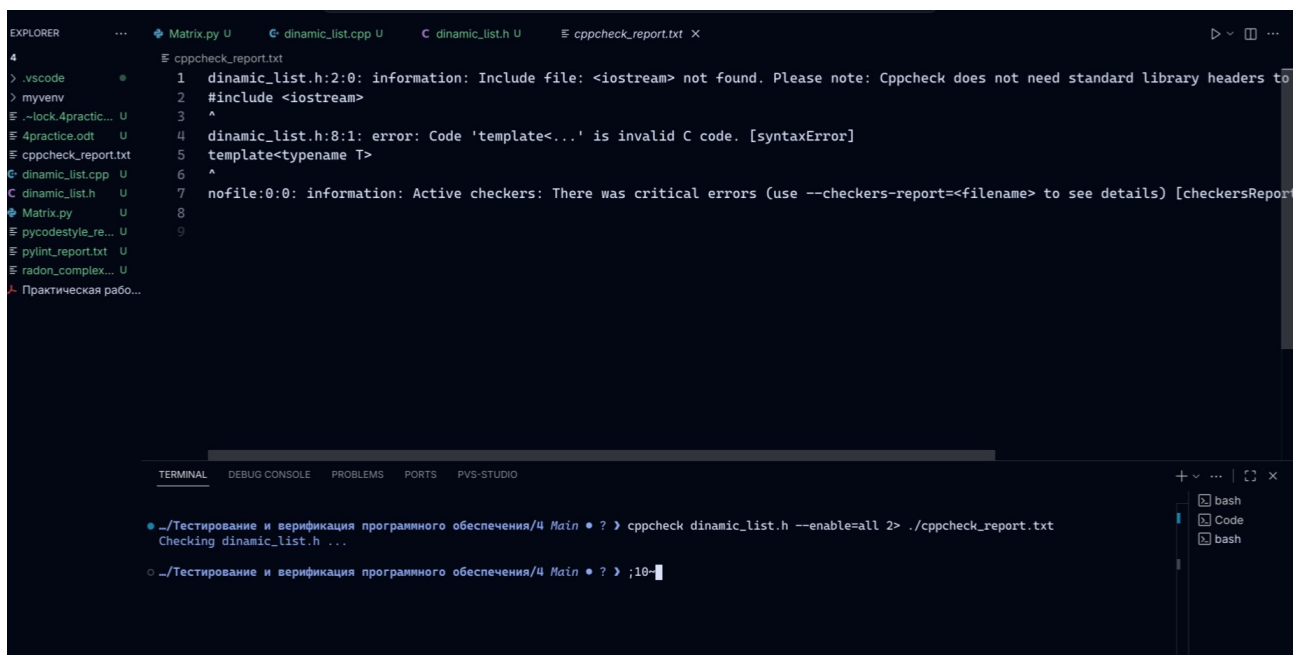


Рисунок 13 - Запуск cppcheck

2.2.5. Статический анализатор проекта «динамический список»(FlawFinder - Поиск уязвимостей)

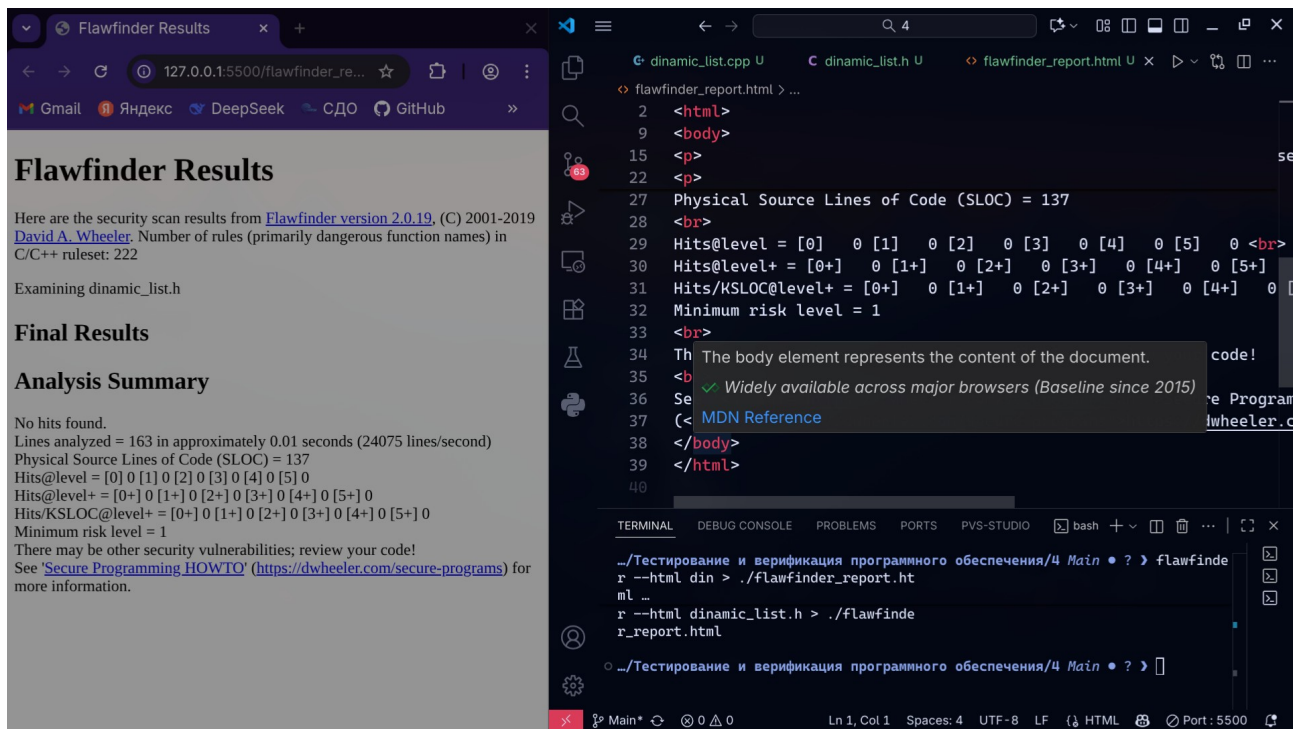


Рисунок 14 - Запуск FlawFinder

2.2.6. Статический анализатор проекта «динамический список»(Cpplint - кодстайл)

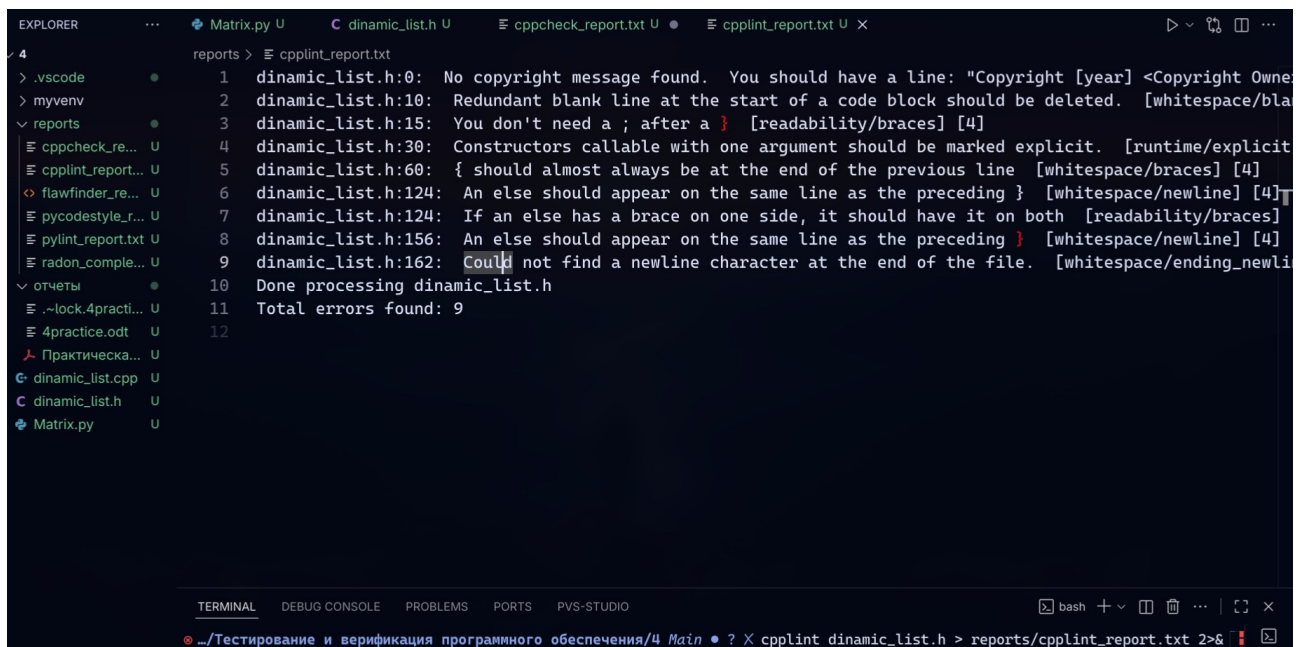


Рисунок 15 - Запуск Cpplint

2.3. Сравнительный анализ

Для python			
Критерий	Pylint	randon	PyCodeStyle
Обнаружено ошибок	17 проблем	9 модулей проанализировано	1 проблема
Ложные срабатывания	Высокий (10+ стилевых)	Низкий (только сложность)	Низкий
Скорость работы	Средняя	Быстрая	Очень быстрая
Простота использования	Простой	Очень простой	Очень простой

Для cpp			
Критерий	Cppcheck	FlawFinder	Cpplint
Обнаружено ошибок	3 проблемы (1 критическая)	0 проблем	9 проблем
Ложные срабатывания	Высокий (ложные ошибки C-кода)	Очень быстрая	Высокий (только стиль)
Скорость работы	Быстрая	Быстрая	Средняя
Простота использования	Средняя	Простой	Сложный

2.4. Внесение ошибок в проекты и повторные статический анализы

2.4.1. Внесение ошибок в проект «Матрица»

```

class Matrix:
    def __init__(self, matrix_v=[], size="1x1"): # PyLint: Опасное значение по умолчанию
        self.matrix_v = matrix_v
        if len(matrix_v) > 0:
            self.size = f"{len(matrix_v)}x{len(matrix_v[0])}"
        else:
            self.size = size
    unused_var = 42 # PyLint: Неиспользуемая переменная

    def change_element(self, index_row, index_col, element):
        if (index_row >= len(self.matrix_v) or
            index_col >= len(self.matrix_v[0])): # PyCodeStyle: Неправильный отступ
            raise IndexError(
                f"Индекс [{index_row}][{index_col}] вне границ матрицы {self.size}" # PyCodeStyle: Слишком длинная строка
            )
        self.matrix_v[index_row][index_col] = element
        x=5 # PyCodeStyle: Отсутствуют пробелы вокруг оператора

```

Рисунок 16 - Внесение ошибок для pylint и pycodestyle

```

64
65 # Radon: Высокая сложность из-за вложенных условий
66 if matrix1.matrix_v:
67     if matrix2.matrix_v:
68         for i in range(len(matrix1.matrix_v)):
69             for j in range(len(matrix1.matrix_v[0])):
70                 if not isinstance(matrix1.matrix_v[i][j], (int, float)):
71                     if not isinstance(matrix2.matrix_v[i][j], (int, float)):
72                         raise TypeError("Элементы должны быть числами")

```

Рисунок 17 - Внесение ошибки для radon

2.4.1.1. Статический анализатор кода - pylint

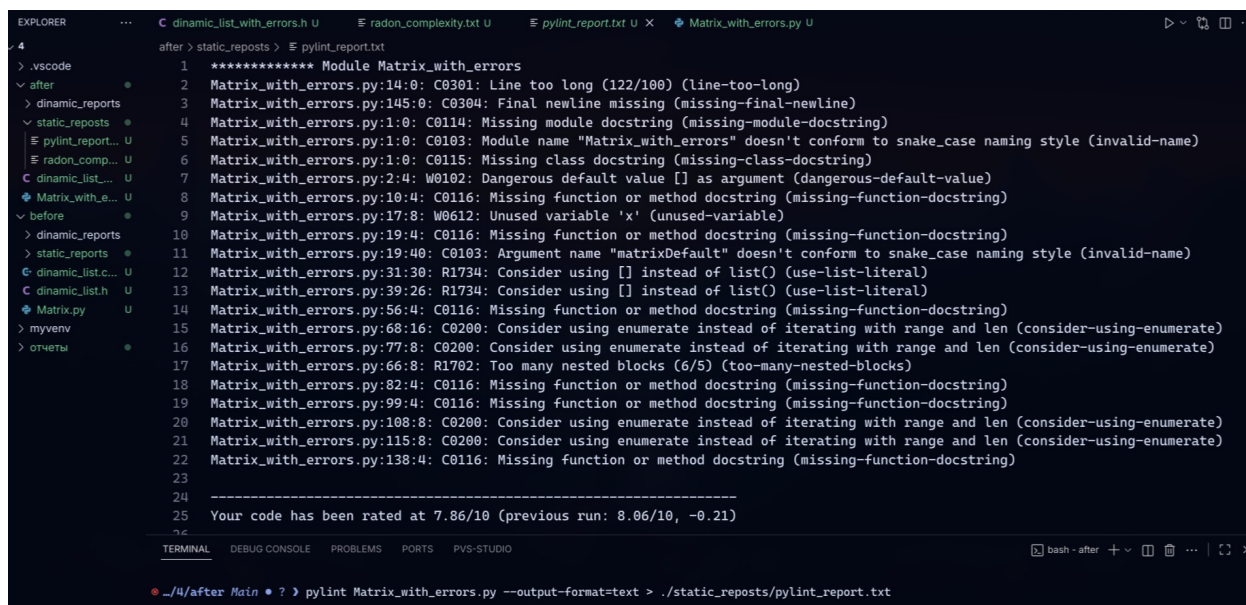


Рисунок 18 - Запуск pylint

2.4.1.2. Статический анализатор кода — radon

```
C dinamic_list_with_errors.h U  radon_complexity.txt U X  pylint_report.txt U  Matrix_with_errors.py U
after > static_reposts > radon_complexity.txt
1  Matrix_with_errors.py
2      M 56:4 Matrix.sum - C (13)
3      M 99:4 Matrix.multiplication - C (13)
4      C 1:0 Matrix - B (7)
5      M 19:4 Matrix.create_matrix - B (7)
6      M 47:4 Matrix._validate_size_format - A (4)
7      M 82:4 Matrix.transposition - A (4)
8      M 10:4 Matrix.change_element - A (3)
9      M 138:4 Matrix.print - A (3)
10     M 2:4 Matrix.__init__ - A (2)
11
```

TERMINAL DEBUG CONSOLE PROBLEMS PORTS PVS-STUDIO

• ../4/after Main • ? > radon cc Matrix_with_errors.py -s > ./static_reposts/radon_complexity.txt

Рисунок 19 - Запуск radon

2.4.1.3. Статический анализатор кода — pycodestyle

```
after > static_reposts > E pycodestyle_report_pep8.txt
42 Matrix_with_errors.py:12:80: E501 line too long (80 > 79 characters)
43     index_col >= len(self.matrix_v[0]): # PyCodeStyle: Неправильный отступ
44
45     Limit all lines to a maximum of 79 characters.
46
47 There are still many devices around that are limited to 80 character
48 lines; plus, limiting windows to 80 characters makes it possible to
49 have several windows side-by-side. The default wrapping on such
50 devices looks ugly. Therefore, please limit all lines to a maximum
51 of 79 characters. For flowing long blocks of text (docstrings or
52 comments), limiting the length to 72 characters is recommended.
53
54 Reports error E501.
55 Matrix_with_errors.py:14:80: E501 line too long (122 > 79 characters)
56     f"Индекс [{index_row}][{index_col}] вне границ матрицы {self.size}" # PyCodeStyle: Слишком длинная строка
57
58     Limit all lines to a maximum of 79 characters.
59
60 There are still many devices around that are limited to 80 character
61 lines; plus, limiting windows to 80 characters makes it possible to
62 have several windows side-by-side. The default wrapping on such
63 devices looks ugly. Therefore, please limit all lines to a maximum
64 of 79 characters. For flowing long blocks of text (docstrings or
65 comments), limiting the length to 72 characters is recommended.
66
67 Reports error E501.
68 Matrix_with_errors.py:17:10: E225 missing whitespace around operator
69     x=5 # PyCodeStyle: Отсутствуют пробелы вокруг оператора
70
TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO  bash - after + v [ ] [ ] ...
~/4/after Main • ? > pycodestyle Matrix_with_errors.py --show-source --show-pep8 > ./static_reposts/pycodestyle_report_pep8.txt
```

Рисунок 20 - Запуск rucodestyle

2.4.2. Внесение ошибок в проект «Динамический список»

```
after > C dinamic_list_with_errors.h > List()
1  #pragma once
2  #include <iostream>
3  #include <cstring> // Сppcheck: Неиспользуемый заголовок
4
```

Рисунок 21 - Добавлена ошибка для сppcheck

```
36  int Size;
37  Node<T> *head;
38  char buffer[100]; // FlawFinder: Фиксированный буфер
39
40  };
41
42  // В отдельный файл нельзя вынести шаблонные методы класса
43
44  template<typename T>
45  List<T>::List() {
46      Size = 0;
47      head = nullptr;
48      strcpy(buffer, "test"); // FlawFinder: Небезопасная функция
49  }
50
```

Рисунок 22 - Добавлены ошибки для flawfinder

```

69     counter++;
70 }
71 // Cppcheck: Возврат ссылки на несуществующий объект
72 static T dummy;
73 return dummy;
74 }
75
76 template<typename T>
77 void List<T>::pop_front() {
78     if (head == nullptr) return;
79     Node<T>* temp = head;
80     head = head->pNext;
81     delete temp;
82     Size--;
83 }
84
85 template<typename T>
86 void List<T>::push_front(T data) {
87     head = new Node<T>(data, head);
88     Size++;
89     int x=5; // Cpplint: Пробел вокруг =
90 }
91

```

Рисунок 23 - Добавлены ошибки для cppcheck и cpluspluslint

```

166     else {
167         maxNodePrev->pNext = maxNode->pNext;
168     }
169
170     prev->pNext = maxNode;
171     maxNode->pNext = nullptr;
172
173     // Cpluspluslint: Длинная строка комментария которая определенно превышает лимит в 80 символов
174 }

```

Рисунок 24 - Добавлена ошибка для cpluspluslint

2.4.2.1. Статический анализатор кода - cpluspluscheck

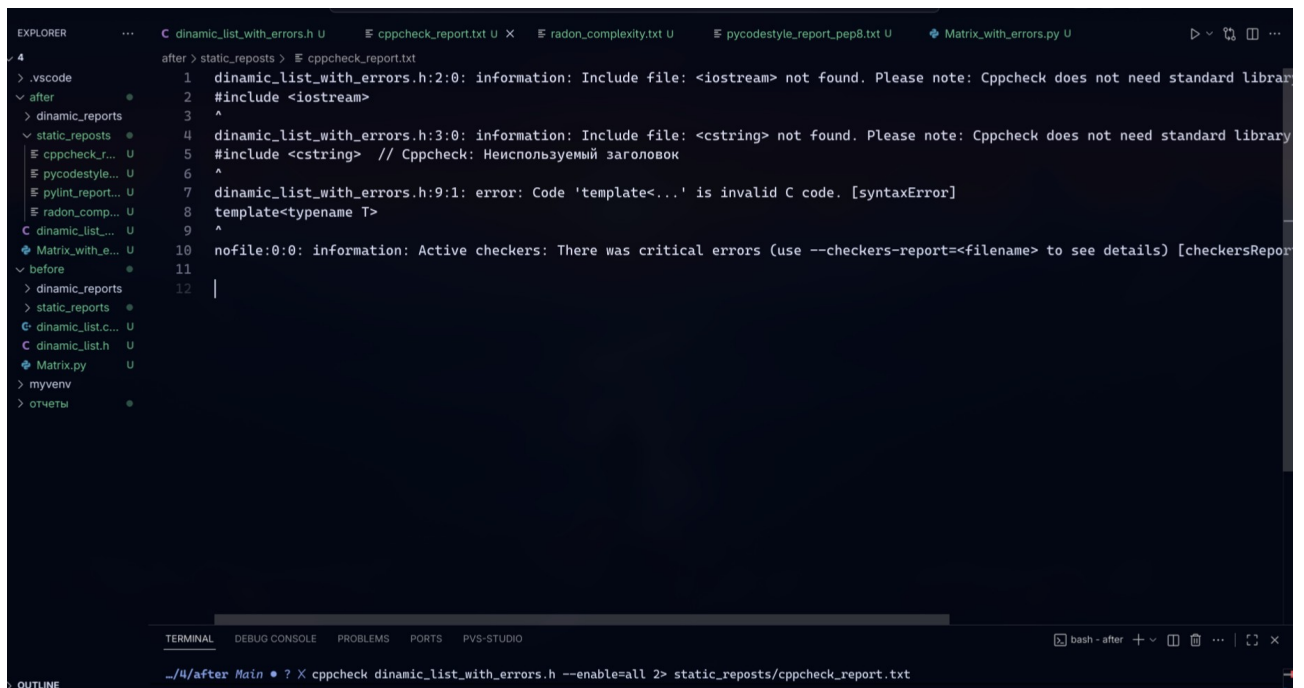


Рисунок 25 - Запуск cppcheck

2.4.2.2. Статический анализатор кода - flawfinder

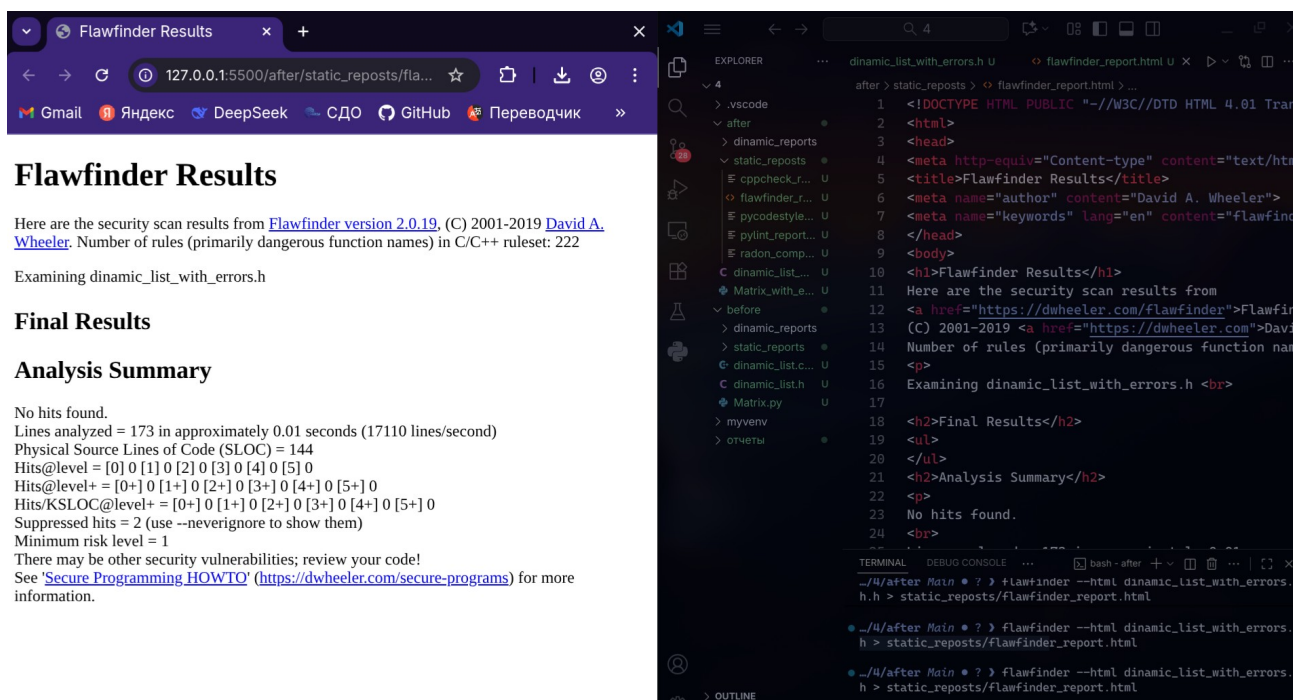


Рисунок 26 - Запуск flawfinder

2.4.2.3. Статический анализатор кода - crrlint

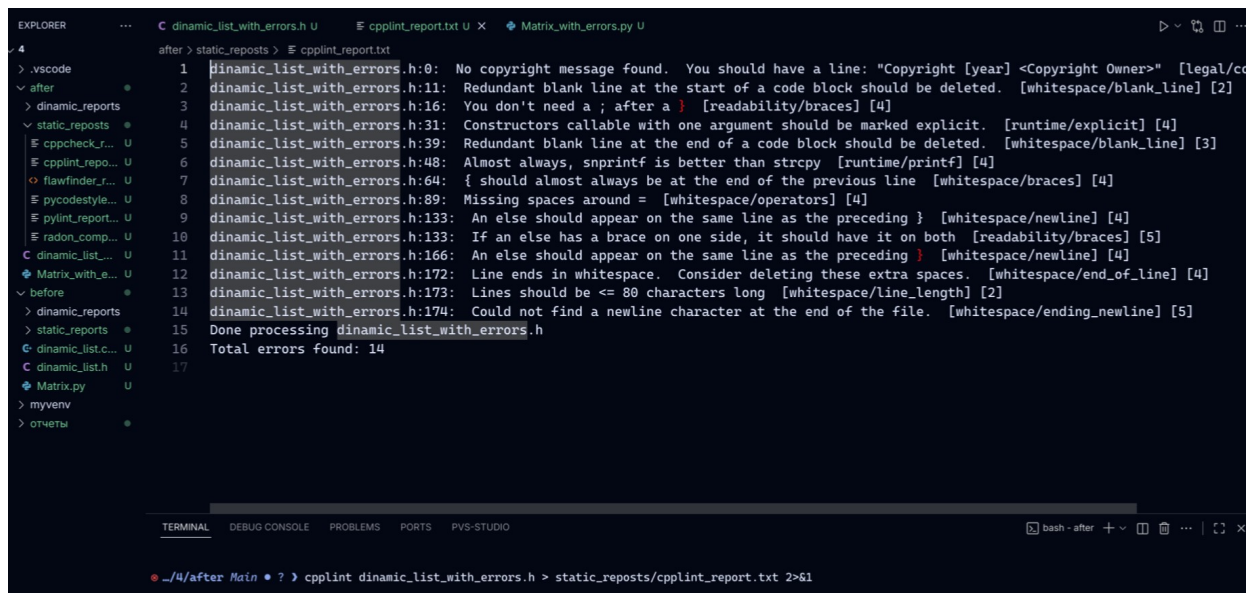


Рисунок 27 - Запуск cpplint

2.5. Повторный сравнительный анализ

Для python

Критерий	Pylint	randon	PyCodeStyle
Обнаружено ошибок	21 проблема	9 модулей	14 проблем
Ложные срабатывания	Высокий (18 стилевых)	Низкий	Высокий (только стиль)
Скорость работы	Средняя	Быстрая	Быстрая
Простота использования	Простой	Очень простой	Очень простой

Для сpp

Критерий	Cppcheck	FlawFinder	Cpplint
Обнаружено ошибок	3 проблемы	0 проблем	14 проблем
Ложные срабатывания	Очень высокий (ложный C- синтаксис)	Нулевой	Очень высокий (только стиль)

Скорость работы	Быстрая	Очень быстрая	Средняя
Простота использования	Средняя	Простой	Сложный

2.6. Вывод об эффективности статического анализа на раннем выявлении ошибок

Статический анализатор позволил нам выявить ошибки до output, что позволило нам эффективнее написать код, избавиться от некрасивого стиля, а также уменьшить сложность реализации каких-то методов. Но при этом какие-то ошибки упускаются. Python-анализаторы (Pylint, PyCodeStyle) нашли 20+ стилевых проблем, C++ анализаторы (Cppcheck, Cpplint) обнаружили синтаксические и стилевые ошибки, но пропустили все реальные баги. Эффективны для поддержания code quality, но недостаточны для комплексной проверки надежности кода. Требуют комбинации с динамическим анализом.

2.7. Динамический анализ кода

При динамическом анализе кода нам понадобится дополнительный файл для запуска основного кода, поэтому добавим их:

```
≡ cProfile_reports.txt U   Matrix.py U   ≡ memory_profile.txt U   Matrix_start.py U ×
before > Matrix_start.py > matrix_operations
1  from Matrix import Matrix
2  ⚡
3  def matrix_operations():
4      m1 = Matrix()
5      m2 = Matrix()
6
7      m1.create_matrix("2x2", True)
8      m2.create_matrix("2x2", True)
9
10     print("Матрица 1:")
11     m1.print()
12
13     print("\nМатрица 2:")
14     m2.print()
15
16     print("\nСложение матриц:")
17     try:
18         result_sum = Matrix.sum(m1, m2)
19         result_sum.print()
20     except Exception as e:
21         print(f"Ошибка сложения: {e}")
22
23     print("\nТранспонирование матрицы 1:")
24     try:
25         transposed = m1.transposition()
26         transposed.print()
27     except Exception as e:
28         print(f"Ошибка транспонирования: {e}")
29
30     print("\nУмножение матриц:")
31     try:
```

Рисунок 28 - Matrix_start.py для запуска динамического анализатора кода на python(часть 1)

```

29
30     print("\nУмножение матриц:")
31     try:
32         result_mul = Matrix.multiplication(m1, m2)
33         result_mul.print()
34     except Exception as e:
35         print(f"Ошибка умножения: {e}")
36
37     print("\nИзменение элемента:")
38     try:
39         m1.change_element(0, 0, 99)
40         m1.print()
41     except Exception as e:
42         print(f"Ошибка изменения: {e}")
43
44 if __name__ == "__main__":
45     matrix_operations()

```

Рисунок 29 - Matrix_start.py для запуска динамического анализатора кода на python(часть 2)

```
valgrind_report.txt U  dynamic_list.cpp U x  dynamic_list U
before > dynamic_list.cpp > ...
1 // test_auto.cpp
2 #include "dynamic_list.h"
3 |
4 int main() {
5     List<int> list;
6
7     list.push_front(3);
8     list.push_front(2);
9     list.push_front(1);
10
11     std::cout << "Size: " <<
12
13     for (int i = 0; i < list.GetSize(); i++) {
14         std::cout << "list[" << i << "] = " << list[i] << std::endl;
15     }
16
17     list.task1(10, 20);
18     list.task2();
19     list.task3();
20
21     list.push_front(-5);
22     list.task2();
23
24     return 0;
25 }
```

```
inline int List<int>::GetSize()
Loading...
```

Рисунок 30 - dynamic_list.cpp для запуска динамического анализатора
кода на с++

2.7.1. Динамический анализатор проекта «Матрица»(сProfile)

```

before > dinamic_reports > cProfile_reports.txt
20
21 Умножение матриц:
22 [7, 10]
23 [15, 22]
24 Размер: 2x2
25
26 Изменение элемента:
27 [99, 2]
28 [3, 4]
29 Размер: 2x2
30 309 function calls (308 primitive calls) in 0.001 seconds
31
32 Ordered by: cumulative time
33
34 nccalls tottime percall cumtime percall filename:lineno(function)
35 2/1 0.000 0.000 0.001 0.001 {built-in method builtins.exec}
36 1 0.000 0.000 0.001 0.001 Matrix_start.py:1(<module>)
37 1 0.000 0.000 0.001 0.001 <frozen importlib._bootstrap>:1349(_find_and_load)
38 1 0.000 0.000 0.001 0.001 <frozen importlib._bootstrap>:1304(_find_and_load_unlocked)
39 1 0.000 0.000 0.001 0.001 <frozen importlib._bootstrap>:911(_load_unlocked)
40 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap_external>:1020(exec_module)
41 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap_external>:1093(get_code)
42 1 0.000 0.000 0.000 0.000 Matrix_start.py:3(matrix_operations)
43 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap>:1240(_find_spec)
44 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap_external>:1551(find_spec)
45 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap_external>:1522(_get_spec)
46 1 0.000 0.000 0.000 0.000 <frozen importlib._bootstrap_external>:1621(find_spec)

```

Рисунок 31 - Запуск cProfile

2.7.2. Динамический анализатор проекта «Матрица»(memory-profiler)

Для этого анализатора нужно будет чуть отредактировать код. Добавим нужную библиотеку, после скачки плагина:

```

from memory_profiler import profile
class Matrix:

```

Рисунок 32 - Библиотека profile

А теперь возьмём из этой библиотек декораторы и добавим их к некоторым методам(в данной работе добавим только к методам нестатическим):


```

@profile
def change_element(self, index_row, index_col, element):
    if (index_row >= len(self.matrix_v) or
        index_col >= len(self.matrix_v[0])):
        raise IndexError(
            f"Индекс [{index_row}][{index_col}] "
            f"вне границ матрицы {self.size}"
        )
    self.matrix_v[index_row][index_col] = element
@profile
def create_matrix(self, size="3x3", matrixDefault=True) -> list:
    if not self._validate_size_format(size):
        raise ValueError(
            f"Неверный формат размера: {size}. Используйте формат 'NxM'"

```

Рисунок 33 - Декораторы для работы библиотеки

```

85     @profile
86     def transposition(self) -> "Matrix":
87         if not self.matrix_v:
88             raise ValueError("Матрица пуста, транспонирование невозможно")

```

Рисунок 34 - Декораторы для работы библиотеки

А теперь запустим динамический анализатор:

The screenshot shows a dynamic analysis tool interface with a table of memory usage and line contents. The table has columns: Line #, Mem usage, Increment, Occurrences, and Line Contents. The data is as follows:

Line #	Mem usage	Increment	Occurrences	Line Contents
18	25.3 MiB	25.3 MiB	1	@profile
19				
20	25.3 MiB	0.0 MiB	1	def create_matrix(self, size="3x3", matrixDefault=True) -> list:
21				if not self._validate_size_format(size):
22				raise ValueError(
23				f"Неверный формат размера: {size}. Используйте формат 'NxM'"
24				)
25	25.3 MiB	0.0 MiB	1	self.size = size
26	25.3 MiB	0.0 MiB	1	self.matrix_v = []
27				
28	25.3 MiB	0.0 MiB	1	if matrixDefault:
29	25.3 MiB	0.0 MiB	1	max_var = 1
30	25.3 MiB	0.0 MiB	3	for i in range(int(self.size[0])):
31	25.3 MiB	0.0 MiB	2	matrix_prom = list()
32	25.3 MiB	0.0 MiB	6	for j in range(int(self.size[2])):
33	25.3 MiB	0.0 MiB	4	matrix_prom.append(max_var)
34	25.3 MiB	0.0 MiB	4	max_var += 1
35	25.3 MiB	0.0 MiB	2	self.matrix_v.append(matrix_prom)
36	25.3 MiB	0.0 MiB	1	return self.matrix_v
37				
38				for i in range(int(self.size[0])):
39				matrix_prom = list()

The terminal at the bottom shows the command: `python -m memory_profiler Matrix_start.py > dynamic_reports/memory_profile.txt 2>&1`

Рисунок 35 - Запуск

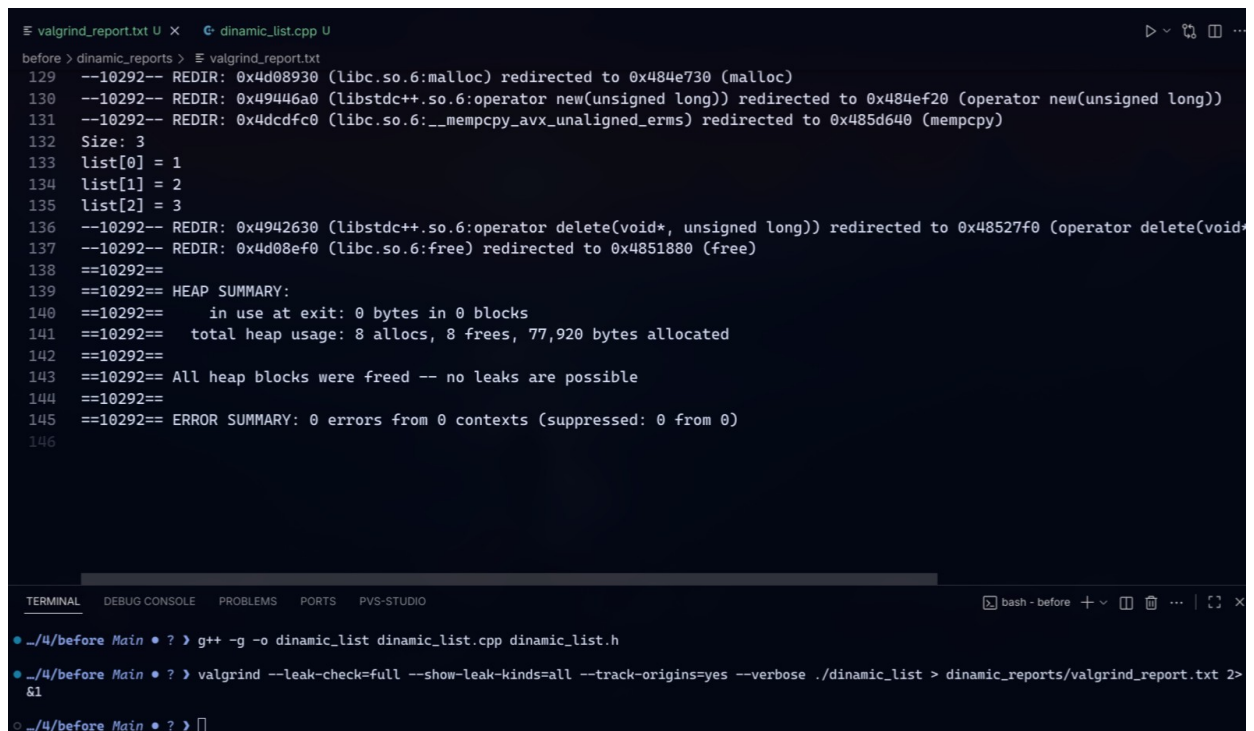
2.7.3. Динамический анализатор проекта «Динамический список»(Valgrind)

При компиляции файла, понял что при операторе [] у меня не обработан вариант при не нахождение элемента. Добавим библиотеку для выбрасывания ошибок: `#include <stdexcept>`. Теперь добавим строчку кода:

```
55     template<typename T>
56     T& List<T>::operator[](const int index) {
57         int counter = 0;
58         Node<T>* current = this->head;
59         while (current != nullptr) {
60             if (counter == index)
61             {
62                 return current->data;
63             }
64
65             current = current->pNext;
66             counter++;
67         }
68
69         throw std::out_of_range("Index out of range");
70     }
```

Рисунок 36 - Добавил throw, для того чтоб убрать предупреждения при
компиляции

Теперь запустим valgrind:

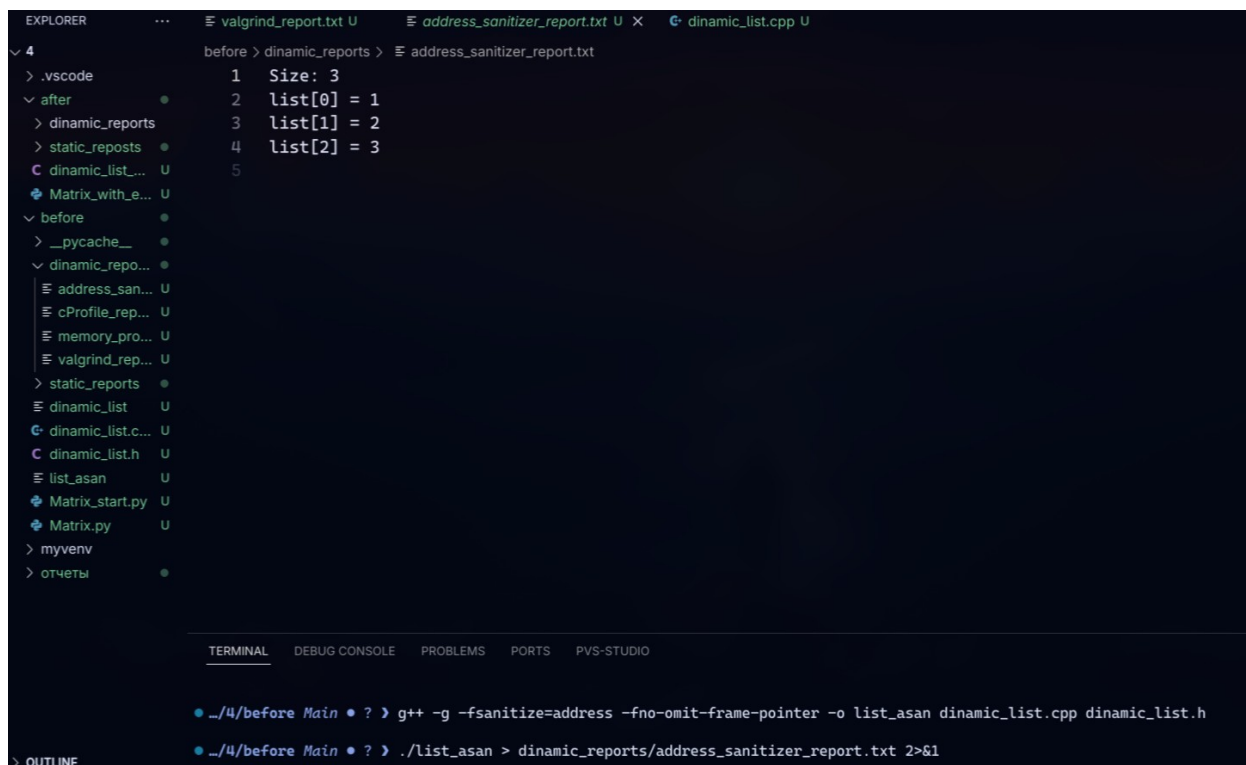


```
valgrind_report.txt X dynamic_list.cpp U
before > dynamic_reports > valgrind_report.txt
129 --10292-- REDIR: 0x4d08930 (libc.so.6:malloc) redirected to 0x484e730 (malloc)
130 --10292-- REDIR: 0x49446a0 (libstdc++.so.6:operator new(unsigned long)) redirected to 0x484ef20 (operator new(unsigned long))
131 --10292-- REDIR: 0x4dcdfc0 (libc.so.6:___mempcpy_avx_unaligned_erms) redirected to 0x485d640 (mempcpy)
132 Size: 3
133 list[0] = 1
134 list[1] = 2
135 list[2] = 3
136 --10292-- REDIR: 0x4942630 (libstdc++.so.6:operator delete(void*, unsigned long)) redirected to 0x48527f0 (operator delete(void*))
137 --10292-- REDIR: 0x4d08ef0 (libc.so.6:free) redirected to 0x4851880 (free)
138 ==10292==
139 ==10292== HEAP SUMMARY:
140 ==10292==      in use at exit: 0 bytes in 0 blocks
141 ==10292==    total heap usage: 8 allocs, 8 frees, 77,920 bytes allocated
142 ==10292==
143 ==10292== All heap blocks were freed -- no leaks are possible
144 ==10292==
145 ==10292== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
146

TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO
bash - before + - - - - -
./4/before Main • ? > g++ -g -o dynamic_list dynamic_list.cpp dynamic_list.h
./4/before Main • ? > valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --verbose ./dynamic_list > dynamic_reports/valgrind_report.txt 2> &1
./4/before Main • ? > |
```

Рисунок 37 - Запуск Valgrind

2.7.4. Динамический анализатор проекта «Динамический список»(AddressSanitizer)



```
EXPLORER  ...  valgrind_report.txt U  address_sanitizer_report.txt U X  dynamic_list.cpp U
4  before > dynamic_reports > address_sanitizer_report.txt
1 Size: 3
2 list[0] = 1
3 list[1] = 2
4 list[2] = 3
5

.vscode
after
dynamic_reports
static_reports
dynamic_list... U
Matrix_with_e... U
before
__pycache__
dynamic_repo...
address_san... U
cProfile_rep... U
memory_pro... U
valgrind_rep... U
static_reports
dynamic_list U
dynamic_list.c... U
dynamic_list.h U
list_asan U
Matrix_start.py U
Matrix.py U
myvenv
отчеты

TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO
./4/before Main • ? > g++ -g -fsanitize=address -fno-omit-frame-pointer -o list_asan dynamic_list.cpp dynamic_list.h
./4/before Main • ? > ./list_asan > dynamic_reports/address_sanitizer_report.txt 2>&1
OUTLINE
```

Рисунок 38 - Запуск AddressSanitizer

2.8. Сравнительный анализ

Для python

Критерий	cProfile	memory-profiler
Обнаружено ошибок	0 проблем	0 проблем
Ложные срабатывания	Нулевой	Нулевой
Скорость работы	Быстрый	Медленный
Простота использования	Простой	Сложный

Для cpp

Критерий	Valgrind	AddressSanitizer
Обнаружено ошибок	0 проблем	0 проблем
Ложные срабатывания	Нулевой	Нулевой
Скорость работы	Очень медленный	Быстрый
Простота использования	Сложный	Простой

2.9. Внесение ошибок в проекты и повторные динамические анализы

2.9.1. Внесение ошибок в проект «Матрица»

```

1  from memory_profiler import profile
2  import time # Для cProfile ошибок
3
4  class Matrix:
5      def __init__(self, matrix_v=[], size="1x1"):
6          self.matrix_v = matrix_v
7          if len(matrix_v) > 0:
8              self.size = f"{len(matrix_v)}x{len(matrix_v[0])}"
9          else:
10             self.size = size
11
12         # Memory Profiler: Утечка через статическую переменную класса
13         if not hasattr(Matrix, 'memory_leak_storage'):
14             Matrix.memory_leak_storage = []
15         Matrix.memory_leak_storage.append(self.matrix_v)
16

```

Рисунок 39 - Добавление ошибок в файл
Matrix_with_errors_for_dyn.py(часть 1)

```

27  @profile
28  def create_matrix(self, size="3x3", matrixDefault=True) -> list:
29      if not self._validate_size_format(size):
30          raise ValueError(
31              f"Неверный формат размера: {size}. Используйте формат 'NxM'"
32          )
33
34      self.size = size
35      self.matrix_v = []
36
37      # Memory Profiler: Избыточное копирование данных
38      temp_lists = []
39      for i in range(int(self.size[0])):
40          matrix_prom = list()
41          for j in range(int(self.size[2])):
42              matrix_prom.append(1)
43              temp_lists.append(matrix_prom[:]) # Лишнее копирование на каждой итерации
44          self.matrix_v.append(matrix_prom)
45      return self.matrix_v
46

```

Рисунок 40 - Добавление ошибок в файл
Matrix_with_errors_for_dyn.py(часть 2)


```

56     @staticmethod
57     def sum(matrix1, matrix2) -> "Matrix":
58         if not matrix1.matrix_v or not matrix2.matrix_v:
59             raise ValueError("Одна из матриц пуста")
60
61         if matrix1.size != matrix2.size:
62             raise ValueError(
63                 f"Размеры матриц не совпадают: {matrix1.size} и {matrix2.size}"
64             )
65
66         # cProfile: Искусственная задержка в цикле
67         result_matrix = []
68         for i in range(len(matrix1.matrix_v)):
69             row = []
70             for j in range(len(matrix1.matrix_v[0])):
71                 time.sleep(0.0001) # Замедление выполнения
72                 row.append(matrix1.matrix_v[i][j] + matrix2.matrix_v[i][j])
73             result_matrix.append(row)
74         return Matrix(result_matrix)
75

```

Рисунок 41 - Добавление ошибок в файл
Matrix_with_errors_for_dyn.py(часть 3)

```

93     @staticmethod
94     def multiplication(matrix1, matrix2) -> "Matrix":
95         if not matrix1.matrix_v or not matrix2.matrix_v:
96             raise ValueError("Одна из матриц пуста")
97
98         if len(matrix1.matrix_v[0]) != len(matrix2.matrix_v):
99             raise ValueError(
100                 f"Несовместимые размеры для умножения: "
101                 f"{matrix1.size} и {matrix2.size}"
102             )
103
104         rows1 = len(matrix1.matrix_v)
105         cols1 = len(matrix1.matrix_v[0])
106         cols2 = len(matrix2.matrix_v[0])
107
108         # cProfile: Лишняя условная проверка в цикле
109         result = []
110         for i in range(rows1):
111             row = []
112             for j in range(cols2):
113                 sum_val = 0
114                 for k in range(cols1):
115                     sum_val += matrix1.matrix_v[i][k] * matrix2.matrix_v[k][j]
116                     if k % 2 == 0: # Бессмысленная проверка
117                         sum_val += 0 # Бессмысленная операция
118                 row.append(sum_val)
119             result.append(row)
120
121         return Matrix(result)
122

```

Рисунок 42 - Добавление ошибок в файл
Matrix_with_errors_for_dyn.py(часть 4)

2.9.1.1. Динамический анализатор кода — cProfile

```
after > dynamic_reports > profile.txt
2
3 Line # Mem usage Increment Occurrences Line Contents
4 =====
5 27 25.0 MiB 25.0 MiB 1 @profile
6 28
7 29 25.0 MiB 0.0 MiB 1 def create_matrix(self, size="3x3", matrixDefault=True) -> list:
8 30 if not self._validate_size_format(size):
9 31 raise ValueError(
10 32 f"Неверный формат размера: {size}. Используйте формат 'NxM'"
11 33 )
12 34 25.0 MiB 0.0 MiB 1 self.size = size
13 35 25.0 MiB 0.0 MiB 1 self.matrix_v = []
14 36
15 37 # Memory Profiler: Избыточное копирование данных
16 38 temp_lists = []
17 39 25.0 MiB 0.0 MiB 3 for i in range(int(self.size[0])):
18 40 25.0 MiB 0.0 MiB 2 matrix_prom = list()
19 41 25.0 MiB 0.0 MiB 6 for j in range(int(self.size[2])):
20 42 25.0 MiB 0.0 MiB 4 matrix_prom.append(1)
21 43 25.0 MiB 0.0 MiB 4 temp_lists.append(matrix_prom[:]) # Лишнее копирование на каждой итерации
22 44 25.0 MiB 0.0 MiB 2 self.matrix_v.append(matrix_prom)
23 45 25.0 MiB 0.0 MiB 1 return self.matrix_v
24
25
26 Filename: /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/M
27
28 Line # Mem usage Increment Occurrences Line Contents
29 =====
30 27 25.1 MiB 25.1 MiB 1 @profile
```

TERMINAL DEBUG CONSOLE PROBLEMS PORTS PVS-STUDIO

• ./after/dynamic_code Main • ? > python -m cProfile Matrix_start.py > ../dynamic_reports/profile.txt

bash after

Code dynamic code

Рисунок 43 - Запуск cProfile

2.9.1.2. Динамический анализатор кода — memory-profiler

```
1 Filename: /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/Matrix_with_errors_for_dyn.py
2
3 Line # Mem usage Increment Occurrences Line Contents
4 =====
5 27 25.1 MiB 25.1 MiB 1 @profile
6 28 def create_matrix(self, size="3x3", matrixDefault=True) -> list:
7 29 25.1 MiB 0.0 MiB 1 if not self._validate_size_format(size):
8 30 raise ValueError(
9 31 f"Неверный формат размера: {size}. Используйте формат 'NxM'"
10 32 )
11 33
12 34 25.1 MiB 0.0 MiB 1 self.size = size
13 35 25.1 MiB 0.0 MiB 1 self.matrix_v = []
14 36
15 37 # Memory Profiler: Избыточное копирование данных
16 38 temp_lists = []
17 39 25.1 MiB 0.0 MiB 3 for i in range(int(self.size[0])):
18 40 25.1 MiB 0.0 MiB 2 matrix_prom = list()
19 41 25.1 MiB 0.0 MiB 6 for j in range(int(self.size[2])):
20 42 25.1 MiB 0.0 MiB 4 matrix_prom.append(1)
21 43 25.1 MiB 0.0 MiB 4 temp_lists.append(matrix_prom[:]) # Лишнее копирование на каждой итерации
22 44 25.1 MiB 0.0 MiB 2 self.matrix_v.append(matrix_prom)
23 45 25.1 MiB 0.0 MiB 1 return self.matrix_v
24
25
26 Filename: /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/Matrix_with_errors_for_dyn.py
27
28 Line # Mem usage Increment Occurrences Line Contents
29 =====
30 27 25.1 MiB 25.1 MiB 1 @profile
31 28 def create_matrix(self, size="3x3", matrixDefault=True) -> list:
32 29 25.1 MiB 0.0 MiB 1 if not self._validate_size_format(size):
33 30 raise ValueError(
34 31 f"Неверный формат размера: {size}. Используйте формат 'NxM'"
35 32 )
36 33
37 34 25.1 MiB 0.0 MiB 1 self.size = size
```

TERMINAL DEBUG CONSOLE PROBLEMS PORTS PVS-STUDIO

./after/dynamic_code Main • ? > python -m memory_profiler Matrix_start.py > ../dynamic_reports/memory_profile.txt >61

bash after

Рисунок 44 - Запуск memory-profiler

2.9.2. Внесение ошибок в проект «Динамический список»

```

43  template<typename T>
44  List<T>::List() {
45      Size = 0;
46      head = nullptr;
47      leak_buffer = new char[100]; // Valgrind: утечка памяти
48      uaf_buffer = new char[50];
49  }
50
51  template<typename T>
52  List<T>::~~List() {
53      while (Size) {
54          pop_front();
55      }
56      // Valgrind: утечка памяти – не освобождаем leak_buffer
57      delete[] uaf_buffer;
58  }
59

```

Рисунок 45 - Добавление ошибок в файл
dynamic_list_with_errors_dyn.cpp(часть 1)

```

83  template<typename T>
84  void List<T>::push_front(T data) {
85      head = new Node<T>(data, head);
86      Size++;
87
88      // AddressSanitizer: использование после освобождения
89      if (Size == 2) {
90          delete[] uaf_buffer;
91          std::strcpy(uaf_buffer, "use after free");
92      }
93  }
94
95  template<typename T>
96  void List<T>::create_list(int n) {
97      T data;
98      for (int i = 0; i < n; i++) {
99          cout << "Введите данные: "; cin >> data;
100         push_front(data);
101     }
102 }
103
104 template<typename T>
105 void List<T>::task1(T a, T b) {
106     // Valgrind: утечка памяти
107     char* local_leak = new char[200];
108     std::strcpy(local_leak, "memory leak in task1");
109
110     Node<T>* current = head;
111     if (current == nullptr || current->pNext == nullptr) {
112         return;
113     }
114     while (current->pNext->pNext != nullptr) {
115         current = current->pNext;
116     }
117     current->pNext = new Node<T>(a, current->pNext);
118     current->pNext->pNext = new Node<T>(b, current->pNext->pNext);
119     Size += 2;
120 }

```

Рисунок 46 - Добавление ошибок в файл
dynamic_list_with_errors_dyn.cpp(часть 2)

```

122  template<typename T>
123  void List<T>::task2() {
124      // AddressSanitizer: переполнение буфера
125      char small_buffer[5];
126      std::strcpy(small_buffer, "buffer overflow test");
127
128      Node<T>* current = head;
129      Node<T>* prev = nullptr;
130      while (current != nullptr && current->data >= 0) {
131          prev = current;
132          current = current->pNext;
133      }
134      if (current != nullptr && prev != nullptr) {
135          prev->pNext = current->pNext;
136          delete current;
137          Size--;
138      }
139      else if (current != nullptr) {
140          pop_front();
141      }
142  }

```

Рисунок 47 - Добавление ошибок в файл
 dinamic_list_with_errors_dyn.cpp(часть 3)

2.9.2.1. Динамический анализатор кода — Valgrind


```
dynamic_list.cpp U  C dynamic_list_with_errors_dyn.h U  E valgrind_report.txt U X
after > dynamic_reports > E valgrind_report.txt
99 ==57343==
100 ==57343== 73,728 bytes in 1 blocks are still reachable in loss record 9 of 9
101 ==57343== at 0x484E7A8: malloc (vg_replace_malloc.c:446)
102 ==57343== by 0x49400EF: pool (eh_alloc.cc:235)
103 ==57343== by 0x49400EF: __static_initialization_and_destruction_0 (eh_alloc.cc:373)
104 ==57343== by 0x49400EF: _GLOBAL__sub_I_eh_alloc.cc (eh_alloc.cc:456)
105 ==57343== by 0x400B2A6: call_init (dl-init.c:74)
106 ==57343== by 0x400B2A6: call_init (dl-init.c:26)
107 ==57343== by 0x400B37C: _dl_init (dl-init.c:121)
108 ==57343== by 0x4023B9F: ??? (in /usr/lib/ld-linux-x86-64.so.2)
109 ==57343==
110 ==57343== LEAK SUMMARY:
111 ==57343== definitely lost: 200 bytes in 1 blocks
112 ==57343== indirectly lost: 0 bytes in 0 blocks
113 ==57343== possibly lost: 0 bytes in 0 blocks
114 ==57343== still reachable: 78,004 bytes in 8 blocks
115 ==57343== suppressed: 0 bytes in 0 blocks
116 ==57343==
117 ==57343== For lists of detected and suppressed errors, rerun with: -s
118 ==57343== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
119

TERMINAL  DEBUG CONSOLE  PROBLEMS  PORTS  PVS-STUDIO

./after/dynamic_code Main * ? X g++ -g -o test_list di.cpp dynamic_list.h
..._cpp

./after/dynamic_code Main * ? X g++ -g -o test_list dynamic_list_with_errors_dyn.h dynamic_list.cpp
..._cpp

dynamic_list_with_errors_dyn.h:125:8: note: destination object 'small_buffer' of size 5
125 | char small_buffer[5];
    |

./after/dynamic_code Main * ? > valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./test_list > ../dynamic_report
s/valgrind_report.txt 2>&1
dynamic_reports/ dynamic_code/ static_code/ static_reports/

* ./after/dynamic_code Main * ? > valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./test_list > ../dynamic_report
s/valgrind_report.txt 2>&1
Aborted (core dumped) valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./test_list > ../dynamic_reports/valgrind_report.txt 2>&1
```

Рисунок 48 - Запуск Valgrind

2.9.2.2. Динамический анализатор кода — AddressSanitizer

```
1 =====
2 ==62610==ERROR: AddressSanitizer: heap-use-after-free on address 0x7b5d61fe0080 at pc 0x7efd6371e1a7 bp 0x7ffe08b231c0 sp 0x7ffe08b22968
3 WRITE of size 15 at 0x7b5d61fe0080 thread T0
4 #0 0x7efd6371e1a6 in memcpy (/usr/src/debug/gcc/gcc/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:115)
5 #1 0x5597af6478e7 in List<int>::push_front(int) /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/d
6 #2 0x5597af647376 in main /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/dynamic_l
7 #3 0x7efd62e27634 (/usr/lib/libc.so.6+0x27634) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
8 #4 0x7efd62e276e8 in __libc_start_main (/usr/lib/libc.so.6+0x276e8) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
9 #5 0x5597af6471e4 in _start (/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/test_l
10
11 0x7b5d61fe0080 is located 0 bytes inside of 50-byte region [0x7b5d61fe0080,0x7b5d61fe00b2)
12 freed by thread T0 here:
13 #0 0x7efd63722605 in operator delete[](void*) /usr/src/debug/gcc/gcc/libsanitizer/asan/asan_new_delete.cpp:146
14 #1 0x5597af6478a3 in List<int>::push_front(int) /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/d
15 #2 0x5597af647376 in main /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/dynamic_l
16 #3 0x7efd62e27634 (/usr/lib/libc.so.6+0x27634) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
17 #4 0x7efd62e276e8 in __libc_start_main (/usr/lib/libc.so.6+0x276e8) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
18 #5 0x5597af6471e4 in _start (/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/test_l
19
20 previously allocated by thread T0 here:
21 #0 0x7efd63721a5d in operator new[](unsigned long) /usr/src/debug/gcc/gcc/libsanitizer/asan/asan_new_delete.cpp:89
22 #1 0x5597af6476ad in List<int>::List() /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_c
23 #2 0x5597af647354 in main /home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/dynamic_l
24 #3 0x7efd62e27634 (/usr/lib/libc.so.6+0x27634) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
25 #4 0x7efd62e276e8 in __libc_start_main (/usr/lib/libc.so.6+0x276e8) (BuildId: 2f722da304c0a508c891285e6840199c35019c8d)
26 #5 0x5597af6471e4 in _start (/home/sergei/Desktop/Rtu_MIREA/3 курс/5 семестр/Тестирование и верификация программного обеспечения/4/after/dynamic_code/test_l
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
22
```

Критерий	cProfile	memory-profiler
Обнаружено ошибок	0 проблем	1 проблема
Ложные срабатывания	Нулевой	Нулевой
Скорость работы	Быстрая	Медленный
Простота использования	Простой	Сложный
Для сpp		
Критерий	Valgrind	AddressSanitizer
Обнаружено ошибок	3 проблемы	1 проблема
Ложные срабатывания	Низкий	Нулевой
Скорость работы	Очень медленная	Быстрый
Простота использования	Сложный	Простой

2.11. Вывод об эффективности динамического анализа для ошибок, возникающих в реальных условиях исполнения

Динамические анализаторы эффективно обнаруживают ошибки времени выполнения: утечки памяти, использование после освобождения, переполнения буферов. С++ анализаторы показали высокую эффективность: Valgrind нашел 3 из 4 ошибок (утечки, use-after-free), AddressSanitizer обнаружил 1 из 2 проблем. Python анализаторы были менее эффективны: memory-profiler выявил 1 из 2 утечек, cProfile не нашел проблем производительности. Для надежного выполнения программ необходима комбинация обоих подходов. Такие анализаторы полезны в низкоуровневых языках программирования, например, с++, в котором отчистки памяти автоматической нет и можно что-то упустить.

3. АНАЛИЗ И ВЫВОДЫ

В ходе выполнения практической работы получили теоретические знания статического и динамического анализа кода, а также применили знания на практике.